Validation avec une Form Request

Une nouvelle organisation pour les règles déjà connues

Formation : développement, sécurisation et optimisation des applications web

Fil rouge « Mes documents »

Cours et travaux dirigés : environ 30 minutes

La validation du jour 2

Le contrôleur vérifie déjà les données reçues

Dans store et update, Laravel vérifie le titre et la catégorie avant l’enregistrement.

title : obligatoire, texte, 100 caractères maximum
category : obligatoire, texte, 50 caractères maximum

Les mêmes règles apparaissent deux fois dans DocumentController.

Qu’est-ce qu’une Form Request ?

Une classe dédiée au contrôle des données entrantes

DocumentRequest rassemble les règles appliquées à la création et à la modification.

Requête reçue **Validation Laravel** **Contrôleur**

Si les données sont valides, le contrôleur poursuit le traitement.

Si elles sont invalides, Laravel renvoie une réponse 422 en JSON.

Les règles dans DocumentRequest

Un seul endroit pour les deux opérations

app/Http/Requests/DocumentRequest.php

```
public function rules(): array
{
    return [
        'title' => ['required', 'string', 'max:100'],
        'category' => ['required', 'string', 'max:50'],
    ];
}
```

authorize() renvoie true à cette étape : les permissions viendront ensuite.

Le contrôleur utilise la requête

Les données ont déjà été validées par Laravel

```
public function store(DocumentRequest $request): JsonResponse
{
    $data = $request->validated();
    $document = Document::create($data);
    return response()->json($document, 201);
}
```

La méthode update utilise également DocumentRequest et validated().

Les routes et les règles de validation restent les mêmes.

Travaux dirigés et vérifications

Appliquer la modification à l'API des documents

1. Créer DocumentRequest avec Artisan dans Docker.

2. Déplacer les règles et adapter le contrôleur.

3. Envoyer un document valide : réponse 201.

4. Envoyer des données invalides : réponse 422.

Le guide Markdown contient les étapes, le code et les commandes.

Sécuriser les documents

Trois étapes, trois vérifications

1 Authentifier

Identifier la personne qui appelle l’API.

2 Autoriser

Limiter chaque document à son propriétaire.

3 Protéger le fichier

Valider le PDF et contrôler son téléchargement.

Authentification et autorisation

Deux questions successives pour une demande

Authentification

Qui fait la demande ?

Exemple : Amina présente un token valide.

Autorisation

Cette personne a-t-elle accès ?

Exemple : le document appartient-il à Amina ?

Une identité valide ne donne pas automatiquement accès à tous les documents.

Session, token API et JWT

Trois notions à distinguer

Session

Le serveur conserve un état. Le navigateur présente généralement un cookie.

Token API

Le client présente un jeton à chaque demande, souvent avec Bearer.

JWT

Un format de jeton pouvant porter des informations et une signature.

Lire le trajet d’un token

La connexion précède les demandes protégées

1 Connexion

Email et mot de passe envoyés à l’API

→

2 Token

Sanctum émet un jeton personnel

→

3 Accès

Le client présente le jeton à l’API

Authorization: Bearer <token>

Le token est un secret. Dans le TD, utilisez-le dans Postman.

Sanctum dans le TD

Créer, utiliser puis révoquer un token

Connexion

Laravel vérifie les identifiants et renvoie un token personnel.

Routes protégées

Le middleware auth:sanctum contrôle l’accès aux documents.

Déconnexion

Le token utilisé est révoqué. Dans cet exercice, il expire aussi après huit heures.

Première vérification

La protection fonctionne avant la gestion des propriétaires

Sans token

GET /api/documents
Réponse attendue : 401

Avec un token valide

GET /api/documents
Réponse attendue : 200

Après révocation, le même token ne permet plus l’accès.

La propriété d’un document

Un document est associé à un utilisateur

Amina crée un document

Laravel associe le document à son compte à partir du token.

Amina consulte sa liste

La recherche porte uniquement sur les documents qui lui appartiennent.

Les anciens documents

Préserver les données existantes

La migration ajoute une colonne user_id pouvant être vide.

Les documents déjà présents restent en base sans propriétaire automatique.

Ils ne figurent dans aucune liste privée tant qu'ils ne sont pas attribués.

Contrôler chaque opération

La protection s’applique à toutes les routes de documents

Liste et lecture

Afficher les documents du propriétaire et retrouver uniquement les siens.

Modification et suppression

Rechercher le document parmi ceux de l’utilisateur connecté.

Accès à un autre document

La correction répond 404 pour un identifiant hors de sa liste.

Comprendre les réponses HTTP

Le statut indique le résultat de la demande

200 / 201	Lecture ou création réussie.
401	Authentification absente ou invalide.
404	Document absent ou inaccessible.
409 / 422	Fichier déjà associé ou données invalides.
429	Trop de tentatives de connexion.

Téléverser un fichier PDF

Ajouter un fichier à un document déjà créé

1 Le client envoie un fichier avec une requête multipart.

2 Laravel vérifie l’utilisateur et les caractéristiques du fichier.

3 Le fichier est enregistré dans le stockage privé.

Dans cet exercice, un seul fichier est associé à chaque document.

Valider un fichier PDF

Plusieurs vérifications côté serveur

Taille et extension

Le TD accepte un fichier de 1 Mo maximum avec l’extension .pdf.

Type MIME détecté

Laravel contrôle le type à partir du contenu du fichier.

Signature binaire

Les cinq premiers octets doivent correspondre à %PDF-.

Ces contrôles ne remplacent pas une analyse antivirus adaptée.

Stockage privé

Séparer le chemin interne de l’accès utilisateur

Le disque local de Laravel stocke les fichiers dans storage/app/private.

Laravel génère un nom de fichier. Le chemin interne reste côté serveur.

Le client utilise une route protégée pour demander le téléchargement.

Téléchargement contrôlé

Le propriétaire reçoit son fichier

1 Le client présente son token.

2 Laravel cherche le document parmi ceux de cet utilisateur.

3 Le serveur renvoie le PDF en téléchargement.

Un autre utilisateur reçoit 404 pour ce document.

TD unique en trois étapes

Vérifier le résultat avant de poursuivre

Étape 1 — Authentification

Comparer une requête sans token et une requête authentifiée.

Étape 2 — Propriété

Créer un document avec Amina et tester l’accès de Karim.

Étape 3 — Fichiers

Envoyer un PDF, refuser un tiers et télécharger le fichier.

Points à retenir

La sécurité suit la demande du client au fichier

Le token identifie l’utilisateur.

La recherche par propriétaire contrôle l’accès au document.

La validation vérifie les caractéristiques du fichier.

Le stockage privé et la route protégée encadrent le téléchargement.